

Architecture & Concepts Deep-Dive - RetailSense AI

This document explains the technical rationale behind the design and technology choices implemented in the RetailSense AI platform.

■ 1. Why Medallion Architecture?

The Medallion Architecture organizes data into three distinct processing layers (Bronze, Silver, Gold), ensuring structure, quality, and incremental refinements.

Layer	Purpose	Alternate Options Considered	Why We Chose Medallion
Bronze	Stores raw data exactly as-is in Delta format.	Raw CSV directories	If downstream transformations contain bugs, having a raw, queryable Bronze layer allows us to reconstruct history without fetching from source databases again.
Silver	Standardizes column types, cleans duplicates, unpivots, and validates integrity.	Direct-to-Gold aggregation	Keeps cleaned data re-usable for other downstream analytics teams (e.g. ad-hoc query analysts) without duplicating cleaning code.
Gold	Highly optimized business aggregations and inputs for ML modeling.	Complex runtime views	Pre-materializes aggregations (Store, Product, State KPIs) so dashboards and API queries load in milliseconds instead of scanning raw tables.

■ 2. Why Delta Lake instead of traditional CSV or raw Parquet?

Delta Lake is an open-source storage layer that brings reliability to data lakes.

- **ACID Transactions:** If a pipeline fails mid-write, Delta rolls back the entire batch. Traditional CSV or Parquet files would leave partial, corrupt, or duplicate files in storage.
- **Time Travel & Versioning:** Allows querying past states of a table (**AS OF VERSION** or **TIMESTAMP**). This is critical for auditing and retraining machine learning models on historical data.

- **Schema Enforcement:** Prevents corrupting clean tables by rejecting writes with modified columns unless explicitly allowed via schema evolution.
 - **Performance (Metadata Speed & Caching):** Unlike raw directories where Spark has to scan directories to find Parquet files, Delta tracks files in a transaction log (`_delta_log`), resulting in significantly faster query plans.
-

■ 3. Why XGBoost Recursive Forecasting?

Forecasting daily unit sales for thousands of product-store combinations is a challenging time-series problem.

- **Why not Prophet?**

Prophet is built for univariate time series and fits a curve for each time-series individually. Fitting Prophet on thousands of store-item combinations is extremely slow, compute-heavy, and cannot capture joint relationships (e.g., how relative prices affect sales across products).

- **Why not LSTMs (Deep Learning)?**

Deep learning models require massive datasets, GPU compute, and complex tuning. They are prone to overfitting on sparse tabular data.

- **Why XGBoost?**

XGBoost is a decision-tree boosting algorithm. It handles non-linear relationships (pricing, calendar events, promotions) exceptionally well. By using a **Recursive Forecasting** approach:

- We train one model to predict $t+1$ based on lags (e.g. Sales at $t-7$, $t-14$, $t-28$).
 - For $t+2$, we feed the model's own $t+1$ forecast back in as the lag feature.
 - This allows us to generate projections for the next 30 days while retaining input features like day-of-week and promotional variables.
-

■ 4. Why Serverless compute instead of Provisioned Clusters?

- **Provisioned clusters** require spinning up VMs, which takes 3-5 minutes, and you pay for the idle time if the VM doesn't shut down immediately.
 - **Serverless compute** starts instantly, automatically scales up or down based on workload, and bills only for the actual seconds the query runs. This is the most cost-effective and low-friction option for learning and running intermittent pipeline jobs.
-

■ 5. Why LLM Summary Injection (KPI Context)?

Sending raw rows or massive tables to LLM APIs (like OpenAI, Grok, or Claude) has major downsides:

1. **Context Window Limits:** Raw data exceeds token limits.

2. **API Cost:** Charging by token makes querying expensive.

3. **Hallucinations:** Large LLMs are poor at doing math or summing thousands of rows dynamically.

Our Approach:

FastAPI queries the Gold Delta tables to build a structured summary text (KPI numbers, top stores list, top categories list) and feeds that highly distilled context, along with the user's question, to the LLM. The LLM acts as an executive analyst, writing clean summaries and suggestions, ensuring speed, accuracy, and low costs.